2 Cloud native patterns and technologies

This chapter covers

- Understanding development principles for cloud native applications
- Building cloud native applications with Spring Boot
- Containerizing applications with Docker and Buildpacks
- Deploying applications to the cloud with Kubernetes
- Introducing patterns and technologies used in the book

The way we design applications for the cloud is different from traditional approaches. The 12-Factor methodology, consisting of best practices and development patterns, is a good starting point for building applications that can be considered cloud native. I'll explain the methodology in the first part of this chapter, and I'll expand on it throughout the book.

Later in this chapter, we'll build a simple Spring Boot application and run it with Java, Docker, and Kubernetes, as shown in figure 2.1. Throughout the book I'll go deeper into each of those topics, so don't worry if something is not entirely clear. This chapter aims to provide you with a mental map of the journey we'll undertake from code to production in a cloud environment, while familiarizing you with patterns and technologies that we'll use in the rest of the book.

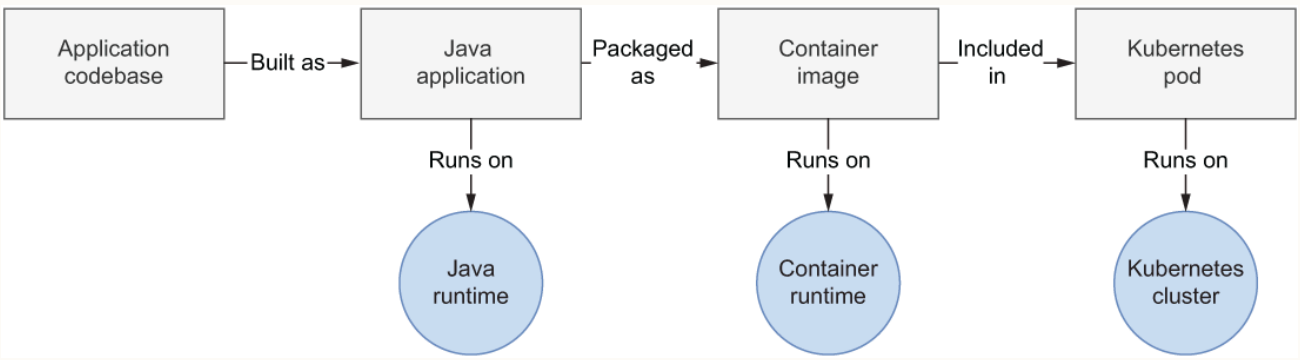


Figure 2.1 The journey of a Spring application from Java to container to Kubernetes

Finally, I'll introduce you to the cloud native project we'll build throughout this book using Spring and Kubernetes. We'll employ all the properties and patterns for cloud native applications introduced in this first part of the book.

2.1 Cloud native development principles: 12 Factors and beyond

The engineers working at the Heroku cloud platform have proposed the 12-Factor methodology as a collection of development principles for designing and building cloud native applications.[1] They have distilled their experience into best practices for building web applications with the following characteristics:

- Suitable to be deployed on cloud platforms
- Scalable by design
- Portable across systems
- Enablers of continuous deployment and agility

The goal was to help developers build applications for the cloud, highlighting the significant factors that should be considered to achieve the best results.

Later, the methodology was revised and expanded by Kevin Hoffman in his book *Beyond the Twelve-Factor App*, refreshing the contents of the original factors and adding three extra ones.[2] From now on, I will refer to this expanded set of principles as the *15-Factor methodology*.

These 15 factors will guide you throughout the book since they are a good starting point for developing cloud native applications. If you are building a new application from scratch or migrating a traditional system to the cloud, these principles can help you on your journey. I will elaborate more on them when relevant, and I'll illustrate how to apply them to Spring applications. It's essential that you become familiar with them.

Let's dig into each of these factors.

1.1 One codebase, one application

The 15-Factor methodology establishes a one-to-one mapping between an application and its codebase, so there's one codebase for each application. Any shared code should be tracked in its own codebase as a library that can be included as a dependency or service that can be run in standalone mode, acting as a backing service for other applications. Each codebase can optionally be tracked in its own repository.

A *deployment* is a running instance of the application. Many deployments are possible in different environments, all sharing the same application artifact. There is no need to rebuild the codebase to deploy an application to a specific environment: any aspect that

changes between deployments (such as configuration) should be outside the application codebase.

1.2 API first

A cloud native system is usually made up of different services that communicate through APIs. Using an *API first* approach while designing a cloud native application encourages you to think about fitting it into a distributed system and favors the distribution of the work across different teams. By designing the API first, another team using that application as a backing service could create their solution against that API. By designing the contract up front, integration with other systems will be more robust and testable as part of the deployment pipeline. Internally, the API implementation can be changed without affecting other applications (and teams) depending on it.

1.3 Dependency management

All application dependencies should be declared explicitly in a manifest and be available for the dependency manager to download from a central repository. In the context of Java applications, we are usually well-equipped to follow this principle using tools like Maven or Gradle. The only implicit dependencies an application can have on the surrounding environment are the language runtime and the dependency manager tool. This means that private dependencies should be resolved via the dependency manager.

1.4 Design, build, release, run

A codebase goes through different stages in its journey from design to deployment in production:

- *Design stage*—Technologies, dependencies, and tools needed by a specific application feature are decided.
- *Build stage*—The codebase is compiled and packaged together with its dependencies as an immutable artifact called a *build*. The build artifact must be uniquely identified.
- *Release stage*—The build is combined with a specific configuration for the deployment. Each release is immutable and should be uniquely identifiable, such as by using semantic versioning (for example, `3.9.4`) or a timestamp (for example, `2022-07-07_17:21`). Releases should be stored in a central repository for easy access, like when a rollback to a previous version is required.
- *Run stage*—The application runs in the execution environment from a specific release.

The 15-Factor methodology requires a strict separation of these stages and doesn't allow changes to the code at runtime, since that would result in a mismatch with the build

stage. The build and the release artifacts should be immutable and labeled with a unique identifier to guarantee reproducibility.

1.5 Configuration, credentials, and code

The 15-Factor methodology defines configuration as everything likely to change between deployments. Whenever you need to change the configuration for an application, you should be able to do so without any changes in the code, and without building the application again.

The configuration might include resource handles to backing services like a database or a messaging system, credentials to access third-party APIs, and feature flags. Ask yourself if any credential or environment-specific information would be compromised should your codebase suddenly become public. That will tell you whether you have correctly externalized the configuration.

To be compliant with this factor, the configuration can't be included in the code or tracked in the same codebase. The only exception is the default configuration, which can be packaged with the application codebase. You can still use configuration files for any other type of configuration, but you should store them in a separate repository.

The methodology recommends storing configuration as environment variables. By doing so, you can have the same application deployed in different environments but with different behaviors depending on the environment's configuration.

1.6 Logs

A cloud native application isn't concerned with routing and storage of logs. Applications should log to the standard output, treating logs as events emitted in a sequence ordered by time. Log storage and rotation are not application responsibilities anymore. An external tool (a *log aggregator*) will fetch, collect, and make logs available for inspection.

1.7 Disposability

In a traditional environment, you would take much care of your applications, ensuring they stay up and running and never terminate. In a cloud environment, you don't need to care that much: applications are ephemeral. If a failure happens and the application doesn't respond anymore, you terminate it and start a new instance. If you have a high-load peak, you can spin up more instances of your applications to sustain the increased workload. We say that an application is disposable if it can be started or stopped at any time.

To handle application instances in such a dynamic way, you should design them to start up quickly whenever you need a new instance and gracefully shut down when you don't need them anymore. A fast startup enables the elasticity of the system, ensuring robustness and resilience. Without a fast startup, you will have performance and availability issues.

A *graceful shutdown* is when an application, on receiving a signal to terminate, stops accepting new requests, completes the ones already in progress, and finally exits. In the case of web processes, that is straightforward. In other cases, such as with worker processes, the jobs they were responsible for must be returned to the work queue, and only afterward can they exit.

1.8 Backing services

Backing services can be defined as external resources that an application uses to deliver its functionality. Examples of backing services are databases, message brokers, caching systems, SMTP servers, FTP servers, or RESTful web services. Treating them as attached resources means that you can easily change them without modifying the application code.

Consider how you use databases throughout the software development life cycle. Chances are that you'll use a different database depending on the stage: development, testing, or production. If you treat the database as an attached resource, you can use a different service depending on the environment. The attachment is done through resource binding. For example, resource binding for a database could consist of a URL, a username, and a password.

1.9 Environment parity

Environment parity is about keeping all your environments as similar as possible. In reality, there are three gaps that this factor tries to address:

- *Time gap*—The period between a code change and its deployment can be quite large. The methodology strives to promote automation and continuous deployment to reduce the period between when a developer writes code to when it's deployed in production.
- *People gap*—Developers build applications, and operators manage their deployment in production. This gap can be resolved by embracing a DevOps culture, improving collaboration between developers and operators, and embracing the "you build it, you run it" philosophy.

- *Tools gap*—One of the main differences between environments is how backing services are handled. For example, developers might use the H2 database in their local environment but PostgreSQL in production. In general, the same type and version of backing services should be used in all environments.

1.10 Administrative processes

Some management tasks are usually needed to support applications. Tasks like database migrations, batch jobs, or maintenance jobs should be treated as one-off processes. Just like application processes, the code for administrative tasks should be tracked in revision control, delivered with the application they support, and executed in the same environment as the application.

It's usually a good idea to frame administrative tasks as small standalone services that run once and then are thrown away or as functions configured in a stateless platform to be triggered when certain events happen, or you can embed them in the application itself, activating them by calling a specific endpoint.

1.11 Port binding

Applications following the 15-Factor methodology should be self-contained and export their services via port binding. In production, there might be some routing services that translate requests from public endpoints to the internal port-bound services.

An application is self-contained if it doesn't depend on an external server in the execution environment. A Java web application would probably run inside a server container like Tomcat, Jetty, or Undertow. A cloud native application, in contrast, would not require the environment to have a Tomcat server available; it would manage it itself as any other dependency. Spring Boot, for example, lets you use an embedded server: the application will contain the server rather than depending on one being available in the execution environment. One of the consequences of this approach is that there is always a one-to-one mapping between application and server, unlike the traditional method where multiple applications are deployed to the same server.

The services provided by the application are then exported via port binding. A web application would bind HTTP services to a specific port and potentially become a backing service for another application. That's what usually happens in a cloud native system.

1.12 Stateless processes

In the previous chapter, you saw that high scalability is one reason why we move to the cloud. To ensure scalability, we design applications as stateless processes and adopt a *share-nothing architecture* : no state should be shared among different application instances. Ask yourself if any data would be lost should an instance of your application be destroyed and recreated. If the answer is affirmative, then your application is not stateless.

No matter what, we will always need to save some state, or our applications will be useless in most cases. As a result, we design applications to be stateless and then only handle the state in specific stateful services like data stores. In other words, a stateless application delegates the state management and storage to a backing service.

1.13 Concurrency

Creating stateless applications is not enough to ensure scalability. If you need to scale, that means you need to serve more users. Therefore, your applications should allow concurrent processing to serve many users at the same time.

The 15-Factor methodology defines processes as first-class citizens. Those processes should be horizontally scalable, distributing the workload across many processes on different machines, and this concurrent processing is only possible if the applications are stateless. In JVM applications, we handle concurrency through multiple threads, available from thread pools.

Processes can be classified according to their types. For example, you might have web processes that handle HTTP requests and worker processes that execute scheduled jobs in the background.

1.14 Telemetry

Observability is one of the properties of cloud native applications. Managing a distributed system in the cloud is complex, and the only way to manage such complexity is by ensuring that every system component provides the correct data to monitor the system's behavior remotely. Examples of telemetry data are logs, metrics, traces, health status, and events. Hoffman uses a very catchy image to stress the importance of telemetry: treat your applications like space probes. What kind of telemetry would you need to monitor and control your applications remotely?

1.15 Authentication and authorization

Security is one of the essential qualities of a software system, but it often doesn't get the necessary attention. Following a *zero-trust* approach, we must secure any interaction within the system at any architectural and infrastructural levels. There is undoubtedly more to security than just authentication and authorization, but those are a good starting point.

With authentication, we can keep track of who is using the application. Knowing that, we can then check the user permissions to verify whether the user is allowed to perform specific actions. A few standards are available for implementing identity and access management, including OAuth 2.1 and OpenID Connect, which we will use in this book.

2.2 Building cloud native applications with Spring

It's time to get more concrete now and start talking about technologies. So far, you have learned about the cloud native approach and the main development practices we'll follow. Now let's have a look at Spring. If you are reading this book, you have probably had some previous experience with Spring, and you'd like to learn how to use it to build cloud native applications.

The Spring ecosystem provides features for handling almost any requirement your applications might have, including those of cloud native applications. Spring is by far the most used Java framework. It's been around for many years, and it's robust and reliable. The community behind Spring is fantastic and willing to move it forward and make it consistently better. Technologies and development practices evolve continuously, and Spring is very good at keeping up with them. Therefore, using Spring for your next cloud native project is an excellent choice.

This section will highlight some interesting features of the Spring landscape. Then we will start to create a Spring Boot application.

2.1 Overview of the Spring landscape

Spring comprises several projects that address many different aspects of software development: web applications, security, data access, integration, batch processing, configuration, messaging, big data, and many more. The beauty of the Spring platform is that it's designed to be modular, so you can use and combine just the projects you need. It doesn't matter which type of application you need to build. Chances are that Spring can help you with it.

Spring Framework is the core of the Spring platform, the project that started it all. It supports dependency injection, transaction management, data access, messaging, web applications, and more. The framework establishes the "plumbing" of enterprise applications so you can focus on the business logic.

Spring Framework provides an execution context (called *Spring context* or *container*), where beans, properties, and resources are managed throughout the entire application life cycle. I'll assume you are already familiar with the framework's core features, so I will not spend too much time on it. In particular, you should be aware of the Spring context's role and be comfortable working with Spring beans, annotation-based configuration, and dependency injection. We will rely on those features, so you should have them sorted out.

Based on the framework, *Spring Boot* makes it possible to build standalone, production-ready applications quickly. Spring Boot takes an opinionated view of Spring and third-party libraries, and it comes bundled with a sensible default configuration that lets developers get started with minimal up-front work while still providing full customization possibilities.

Throughout this book, you'll have a chance to use several Spring projects to implement patterns and best practices for cloud native applications, including Spring Boot, Spring Cloud, Spring Data, Spring Security, Spring Session, and Spring Native.

NOTE If you are interested in learning more about the core Spring features, you can find a few books on the subject in the Manning catalog, including *Spring Start Here* by Laurențiu Spilcă (Manning, 2021) and *Spring in Action*, sixth edition, by Craig Walls (Manning, 2022). You can also refer to *Spring Boot: Up & Running* by Mark Heckler (O'Reilly, 2021).

2.2 Building a Spring Boot application

Imagine that you have been hired to build a Polar Bookshop application for Polarsophia. This organization manages a specialized bookshop and wants to sell its books about the North Pole and the Arctic online. A cloud native approach is being considered.

As a pilot project, your boss has assigned you to demonstrate to your colleagues how to go from implementation to production in the cloud. The web application you are asked to build is Catalog Service, and for now it will only have one responsibility: welcoming users to the book catalog. This pilot project will be the foundation for actual products built as cloud native applications, should it be successful and well-received.

Considering the task's goal, you might decide to implement the application as a RESTful service with a single HTTP endpoint responsible for returning a welcome message. Surprisingly enough, you choose to adopt Spring as the primary technology stack for the one service (Catalog Service) composing the application. The architecture of the system is shown in figure 2.2, and you'll be trying your hand at building and deploying the application in the upcoming sections.

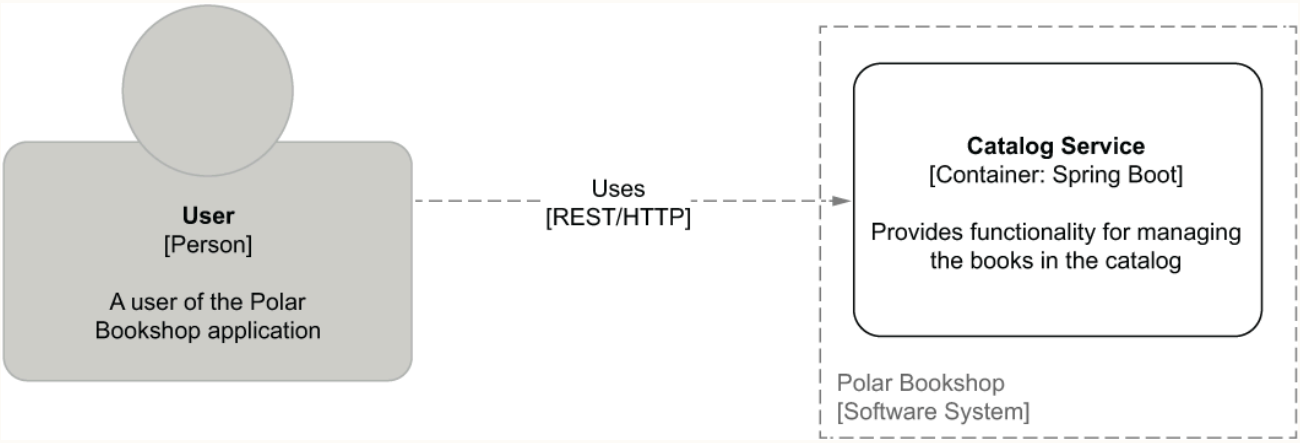


Figure 2.2 Architecture diagram for the Polar Bookshop application, following the C4 model

In figure 2.2, you can see the notation that I'll use to represent architectural diagrams throughout the book, following the C4 model created by Simon Brown (**https://c4model.com**). To describe the architecture for the Polar Bookshop project, I'm relying on three abstractions from the model:

- *Person*—This represents one of the human users of the software system. In our example, it's a customer of the bookshop.
- *System*—This represents the overall application you will build to deliver value to its users. In our example, it's the Polar Bookshop system.
- *Container*—This represents a service, either application or data. It's not to be confused with Docker. In our example, it's the Catalog Service.

For this task we'll use the Spring Framework and Spring Boot to do the following:

- Declare the dependencies needed to implement the application.
- Bootstrap the application with Spring Boot.
- Implement a controller to expose an HTTP endpoint for returning a welcome message.
- Run and try the application.

All the examples in this book are based on Java 17, the latest long-term release of Java at the time of writing. Before moving on, follow the instructions in section A.1 of appen-

dix A to install an OpenJDK 17 distribution. Then ensure you have an IDE with support for Java, Gradle, and Spring. I'll be using IntelliJ IDEA, but feel free to choose another one, such as Visual Studio Code. Finally, create a free account on GitHub if you don't have one yet (**https://github.com**). You'll use it to store your code and define continuous delivery pipelines.

INITIALIZING THE PROJECT

Throughout the book, we'll build several cloud native applications. I recommend you define a Git repository for each of them and use GitHub to store them. In the next chapter, I'll talk more about managing codebases. For now, go ahead and create a catalog-service Git repository.

Next, you can generate the project from Spring Initializr (**https://start.spring.io**) and store it in the catalog-service Git repository you just created. Spring Initializr is a convenient service you can use from a browser or through its REST API to generate JVM-based projects. It's even integrated into popular IDEs such as IntelliJ IDEA and Visual Studio Code. The initialization parameters for Catalog Service are shown in figure 2.3.

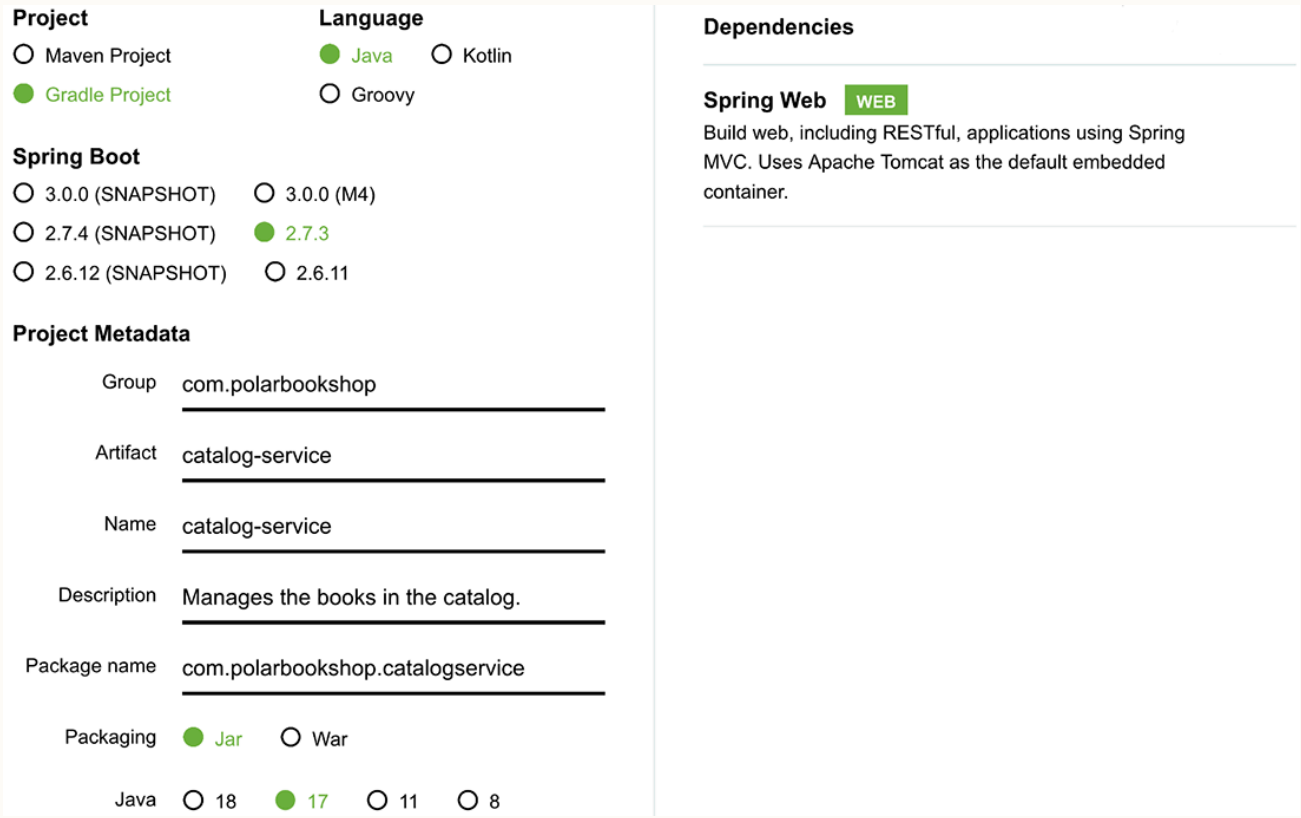


Figure 2.3 The parameters for initializing the Catalog Service project from Spring Initializr

During the initialization, you can provide a few details about the application you want to build, as shown in table 2.1.

Table 2.1 The main parameters you can configure to generate a project from Spring

Initializr

Parameter	Description	Value for Catalog Service
Project	You can decide whether you want to use Gradle or Maven as the build tool for your project. All the examples in this book will use Gradle.	Gradle
Language	Spring supports the three main JVM languages: Java, Kotlin, and Groovy. All the examples in this book will use Java.	Java
Spring Boot	You can choose which version of Spring Boot you want to use. All the examples in this book will use Spring Boot 2.7.3, but any subsequent patch should be fine.	Spring Boot 2.7.3
Group	The group ID for the project, as used in a Maven repository.	com.polarbookshop
Artifact	The artifact ID for the project, as used in a Maven repository.	catalog-service
Name	The project name.	catalog-service
Package name	The base Java package for the project.	com.polarbookshop.catalogservice
Packaging	How to package the project: WAR (for deployment on application servers) or JAR (for standalone applications). Cloud native applications should be packaged as JARs, so all the examples in this book will use that option.	JAR
Java	The Java version you want to use to build the project. All the examples in this book will use Java 17.	17

Dependencies	Which dependencies to include in the project.	Spring Web

The structure of the newly generated project is shown in figure 2.4. I'll guide you through it in the upcoming sections.

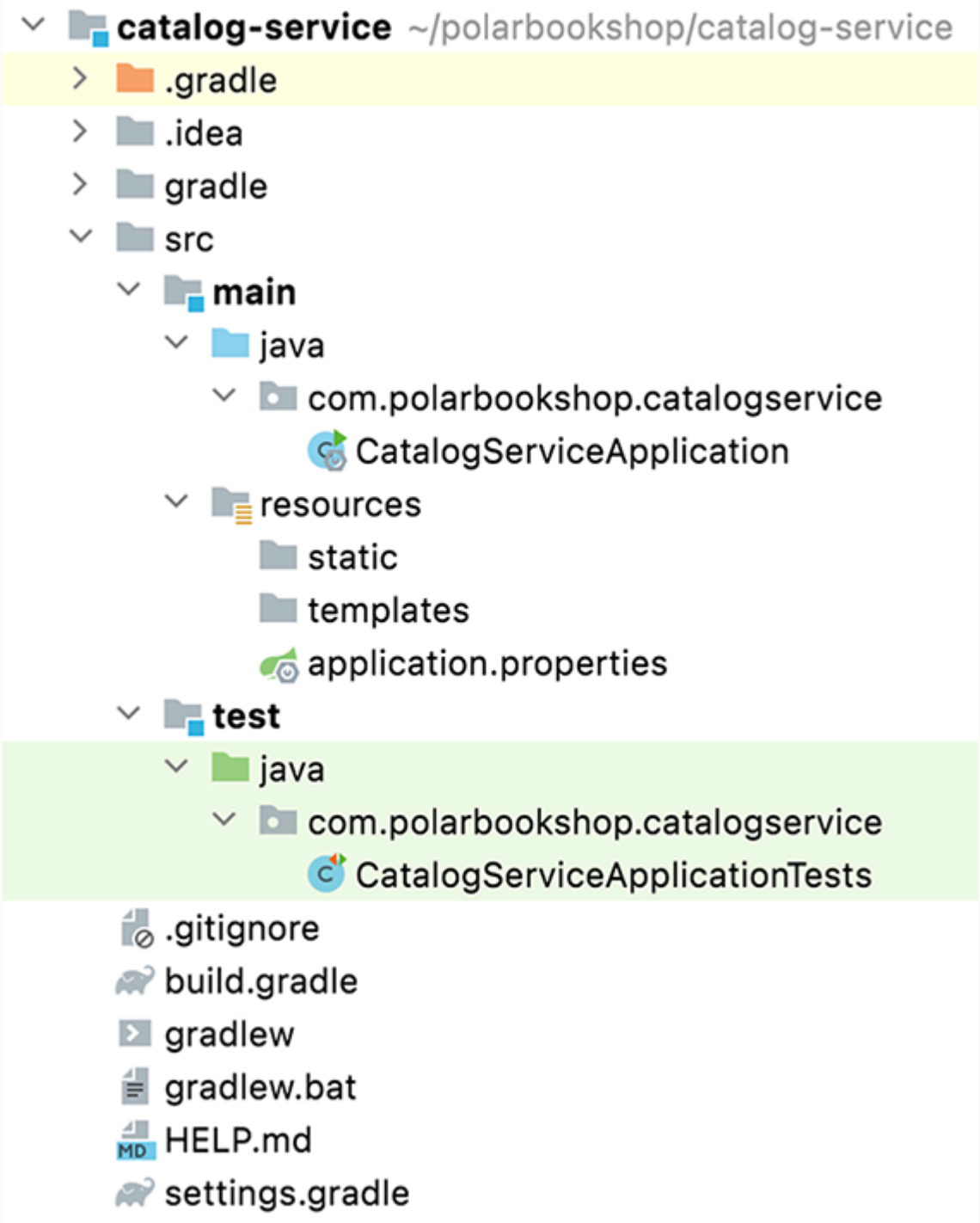


Figure 2.4 The structure of a new Spring Boot project generated from Spring Initializr

In the code repository accompanying this book (https://github.com/ThomasVitale/cloud-native-spring-in-action), you can find "begin" and "end" folders for each chapter, so that you can always start with the same

setup as me and check the final result. For example, you are currently reading chapter 2, so you will find the related code in Chapter02/02-begin and Chapter02/02-end.

TIP In the "begin" folder for this chapter, you'll find a `curl` command that you can run in a Terminal window to download a zip file containing all the code you need to get started, without going through the manual project generation on the Spring Initializr website.

Gradle or Maven?

I'm using Gradle in this book, but feel free to use Maven instead. In the code repository accompanying this book, you can find a table mapping the Gradle commands to Maven so that you can easily follow along, should you use the second option (**https://github.com/ThomasVitale/cloud-native-spring-in-action**). Each project has different needs that might lead you to choose one build tool over another.

My choice of using Gradle is a personal preference and is due to two main reasons. Building and testing a Java project with Gradle takes less time than Maven, thanks to its incremental and parallel builds and caching system. Also, I find the Gradle build language (Gradle DSL) to be more readable, expressive, and maintainable than Maven XML. In the Spring ecosystem, you can find projects using Gradle and others using Maven. They're both good choices. I recommend you try them both and pick the tool that makes you more productive.

EXPLORING THE BUILD CONFIGURATION

Open the project you have just initialized in your favorite IDE, and look at the Gradle build configuration for the Catalog Service application, defined in the build.gradle file. You can find there all the information you provided to Spring Initializr.

Listing 2.1 The build configuration for Catalog Service

```
plugins {
  id 'org.springframework.boot' version '2.7.3'    (1)
  id 'io.spring.dependency-management'
➥ version '1.0.13.RELEASE'                        (2)
  id 'java'                                        (3)
}

group = 'com.polarbookshop'                        (4)
```

```
version = '0.0.1-SNAPSHOT'                    ❺
sourceCompatibility = '17'                    ❻

repositories {                                ❼
  mavenCentral()
}

dependencies {                                ❽
  implementation 'org.springframework.boot:spring-boot-starter-web'
  testImplementation 'org.springframework.boot:spring-boot-starter-test'
}

tasks.named('test') {
  useJUnitPlatform()                          ❾
}
```

❶ Provides Spring Boot support in Gradle and declares which version to use

❷ Provides dependency management features for Spring

❸ Provides Java support in Gradle, setting up tasks to compile, build, and test the application

❹ The group ID for the Catalog Service project

❺ The version of the application. By default, it's 0.0.1-SNAPSHOT.

❻ The Java version used to build the project

❼ Artifact repositories to search for dependencies

❽ The dependencies used by the application

❾ Enables testing with the JUnit Platform provided by JUnit 5

The project contains the following main dependencies:

- *Spring Web* (org.springframework.boot:spring-boot-starter-web) provides the necessary libraries for building web applications with Spring MVC and includes Tomcat as the default embedded server.

- *Spring Boot Test* (org.springframework.boot:spring-boot-starter-test) provides several libraries and utilities for testing applications, including Spring Test, JUnit, AssertJ, and Mockito. It's automatically included in every Spring Boot project.

NOTE Spring Boot offers convenient starter dependencies that bundle together all the libraries necessary for a specific use case, taking care to choose versions that are compatible with each other. This feature significantly simplifies your build configuration.

The name of the project is defined in a second file called settings.gradle:

```
rootProject.name = 'catalog-service'
```

BOOTSTRAPPING THE APPLICATION

In a previous section, you initialized the Catalog Service project and chose the JAR packaging option. Any Java application packaged as a JAR must have a public static void main(String[] args) method that is executed at startup, and Spring Boot is no different. In Catalog Service, a CatalogServiceApplication class was autogenerated during the initialization; that's where the main() method is defined, and it's how the Spring Boot application is run.

Listing 2.2 The bootstrap class for Catalog Service

```
package com.polarbookshop.catalogservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication                    ❶
public class CatalogServiceApplication {
  public static void main(String[] args) {    ❷
    SpringApplication.run(CatalogServiceApplication.class, args);
  }
}
```

❶ Defines a Spring configuration class and triggers component scanning and Spring Boot auto-configuration

❷ The method used to launch the application. It registers the current class to be run during the application's bootstrap phase.

The @SpringBootApplication annotation is a shortcut that includes three different annotations:

- @Configuration marks the class as a source of beans definitions.
- @ComponentScan enables component scanning to find and register beans in the Spring context automatically.
- @EnableAutoConfiguration enables the auto-configuration capabilities offered by Spring Boot.

Spring Boot auto-configuration is triggered by several conditions, such as the presence of certain classes in the classpath, the existence of specific beans, or the values of some properties. Since the Catalog Service project depends on spring-boot-starter-web, Spring Boot will initialize an embedded Tomcat server instance and apply the minimal configuration required to get a web application up and running in almost zero time.

That's it for the application setup. Let's go ahead and expose an HTTP endpoint from Catalog Service.

IMPLEMENTING THE CONTROLLER

So far, we have looked at the project as generated by Spring Initializr. It's time to implement the business logic for the application.

Catalog Service will expose an HTTP GET endpoint to return a friendly greeting to the user to welcome them to the book catalog. You can define a handler for it in a controller class. Figure 2.5 shows the interaction flow.

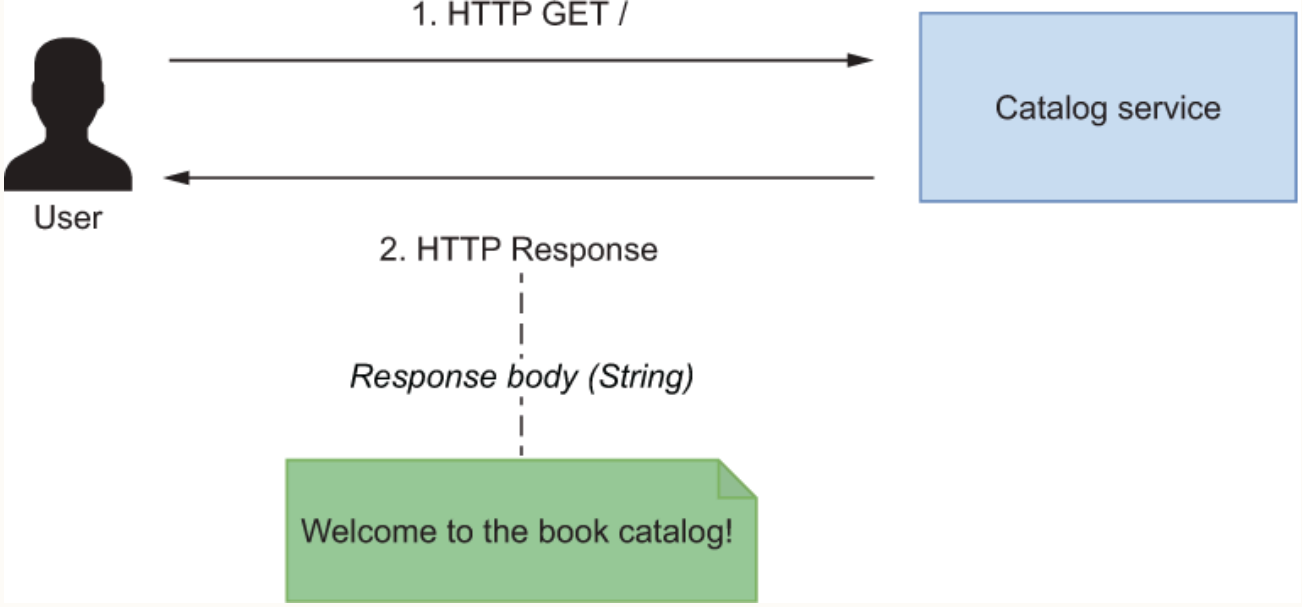


Figure 2.5 Interaction between user and application to get a welcome message from the HTTP endpoint exposed by Catalog Service

In the Catalog Service project, create a new HomeController class, and implement a method responsible for handling GET requests to the root endpoint (/).

Listing 2.3 Defining an HTTP endpoint to return a welcome message

```
package com.polarbookshop.catalogservice;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController                    (1)
public class HomeController {

  @GetMapping("/")                 (2)
  public String getGreeting() {
    return "Welcome to the book catalog!";
  }
}
```

(1) Identifies a class defining handlers for REST/HTTP endpoints

(2) Handles GET requests to the root endpoint

The @RestController annotation identifies a class as a controller handling incoming HTTP requests. Using the @GetMapping annotation, you can mark the getGreeting() method as a handler for GET requests arriving at the root endpoint (/). Any GET

request to that endpoint will be processed by this method. In the next chapter, I'll cover how to build RESTful services with Spring in more detail.

TESTING THE APPLICATION

A basic test setup is included when you create a Spring project from Spring Initializr. In the build.gradle file you automatically get the dependencies needed for testing a Spring application. Furthermore, a test class is automatically generated. Let's look at what the CatalogServiceApplicationTests class might look like after you initialize the project.

Listing 2.4 The auto-generated test class verifying the Spring context

```
package com.polarbookshop.catalogservice;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest                    (1)
class CatalogServiceApplicationTests {

  @Test                            (2)
  void contextLoads() {            (3)
  }
}
```

(1) Provides a setup for testing Spring Boot applications

(2) Identifies a test case

(3) Empty test used to verify that the application context is loaded correctly

The default test class is identified by the @SpringBootTest annotation, which provides many helpful features for testing Spring Boot applications. I'll present them in more detail throughout the book. For now, it's enough to know that it loads a full Spring application context for the tests to be run in. There is currently only one test case, which is empty: it's used to verify that the Spring context is loaded correctly.

Open a Terminal window, navigate to the application root folder (catalog-service), and run the test Gradle task to execute the application's tests.

```
$ ./gradlew test
```

The task should be successful and the test green, meaning that the Spring application can start up without errors. What about the HTTP endpoint? Let's find out.

RUNNING THE APPLICATION

You are done implementing the application, so you can go ahead and run it. There are different ways to do that, and I will show you some of them later. For now, you can use the task provided by the Spring Boot Gradle plugin: bootRun.

From the same Terminal window where you launched the tests, run this command:

```
$ ./gradlew bootRun
```

The application should be up and running in a second and ready to accept requests. In figure 2.6 you can see the logs streamed during the startup phase.

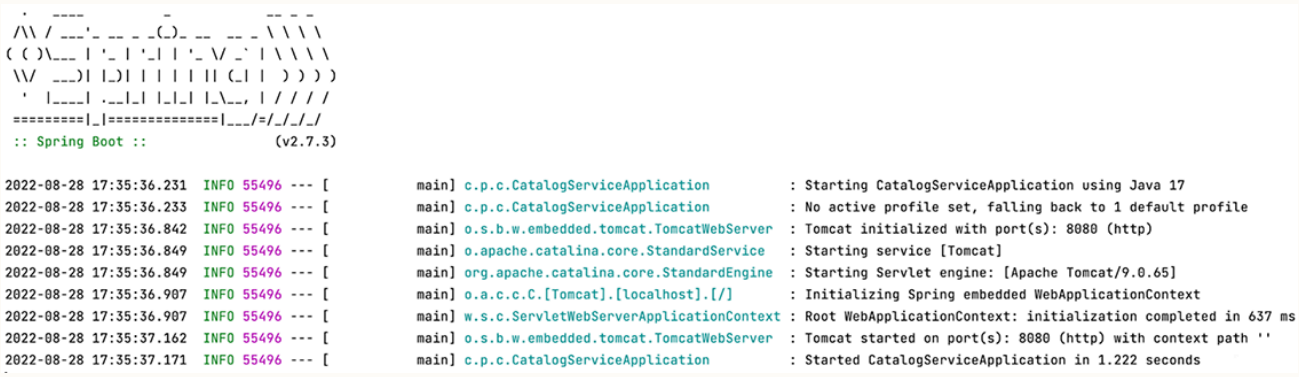


Figure 2.6 The startup logs from the Catalog Service application

From the logs in figure 2.6, you'll notice that the startup phase is composed of two main steps:

- Initialization and running of the embedded Tomcat server (which, by default, listens to port 8080 over HTTP)
- Initialization and running of the Spring application context

At this point, you can finally verify whether your HTTP endpoint is working as intended. Open a browser window, navigate to http://localhost:8080/, and be ready to be welcomed to the Polar Bookshop's book catalog.

```
Welcome to the book catalog!
```

The development part of the Polar Bookshop application is done: you have a Catalog Service application welcoming users to the book catalog. Remember to terminate the `bootRun` process (Ctrl-C) to stop the application execution before moving on.

The next step is deploying the application to the cloud. To make it portable on any cloud infrastructure, you should containerize it first. Enter Docker.

2.3 Containerizing applications with Docker

The Catalog Service application is working. Before deploying it to the cloud, however, you should containerize it. Why? Containers provide isolation from the surrounding environment, and they're equipped with all the dependencies required by the application to run.

In our case, most of the dependencies are managed by Gradle and are packaged together with the application (JAR artifact). But the Java runtime is not included. Without a container, you would have to install the Java runtime on any machine where you want to deploy the application. Containerizing the application means it will be self-contained and portable across any cloud environment. With containers you can manage all applications in a standard way, no matter the language or framework used to implement them.

The *Open Container Initiative* (OCI), a Linux Foundation project, defines industry standards for working with containers (**https://opencontainers.org**). In particular, the OCI Image Specification defines how to build container images, the OCI Runtime Specification defines how to run those container images, and the OCI Distribution Specification defines how to distribute them. The tool we'll use to work with containers is Docker (**www.docker.com**), which is compliant with the OCI specifications.

Docker is an open source platform that "provides the ability to package and run an application in a loosely isolated environment called a container" (**https://docs.docker.com**). *Docker* is also the name of the company behind this technology, which is a founding member of the OCI. The same term is used in several of their commercial products, too. Whenever I write *Docker*, unless specified otherwise, I am referring to the open source platform we'll use to build and run containers.

Before moving on, follow the instructions in section A.2 of appendix A to install and configure Docker in your development environment.

3.1 Introducing Docker: Images and containers

When you install the Docker platform on your machine, you get the Docker Engine package characterized by a client/server architecture. The *Docker server* contains the *Docker daemon*, a background process responsible for creating and managing Docker objects like images, containers, volumes, and networks. The machine where the Docker server runs is called the *Docker host*. Each machine where you want to run containers should be a Docker host, so it should have a Docker daemon running. The portability of containers is made possible by the daemon process itself.

The Docker daemon exposes an API you can use to send instructions, such as to run a container or create a volume. The *Docker client* talks to the daemon through that API. The client is command-line based and can be used to interact with the Docker daemon either through scripting (for example, Docker Compose) or through the Docker CLI directly.

Besides the client and server components characterizing the Docker Engine, another essential element of the platform is a *container registry*, which has a similar function to a Maven repository. While Maven repositories are used to host and distribute Java libraries, container registries do the same for container images, and they follow the OCI Distribution Specification. We distinguish between public and private registries. The Docker company provides a public registry called Docker Hub (**https://hub.docker.com**), which is configured by default with your local Docker installation and hosts images for many popular open source projects, like Ubuntu, PostgreSQL, and OpenJDK.

Based on the architecture description included in the Docker documentation (**https://docs.docker.com**), figure 2.7 shows how the Docker client, Docker server, and container registry interact.

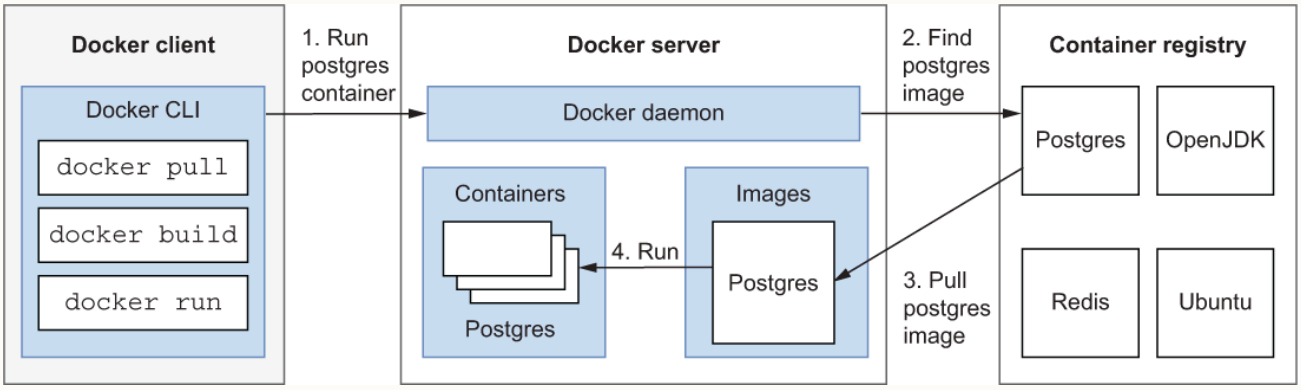


Figure 2.7 The Docker Engine has a client/server architecture, and it interacts with a registry.

The Docker daemon manages different objects. For now we'll focus on images and containers.

A *container image* (or, simply, an *image*) is a lightweight executable package that includes everything needed to run the application inside. The *Docker image* format is the most used one for creating container images, and it has been standardized by the OCI project (in the OCI Image Specification). OCI images can be created from scratch by defining instructions in a Dockerfile, a text-based file containing all the steps to generate the image. Usually, images are created based on another image. For example, you might build an image based on the OpenJDK one, on top of which you can add a Java application. After it's created, the image can be pushed to a container registry like Docker Hub. A base name and a tag identify each image, where the tag is usually the version number. For example, the Ubuntu image for version 22.04 is called ubuntu:22.04. A colon separates the base name and version.

A *container* is a runnable instance of a container image. You can manage the container life cycle from the Docker CLI or Docker Compose: you can start, stop, update, and delete containers. Containers are defined by the image on which they are based and the configuration provided at startup time (for example, environment variables used to customize the container). By default, containers are isolated from each other and the host machine, but you can make them expose services to the outside world through specific ports with a process called *port forwarding* or *port mapping*. Containers can have any name. If you don't specify one, the Docker server will assign a random one, like bazinga_schrodinger. To run OCI images as containers, you'll need Docker or any other container runtime compatible with the OCI specifications.

When you want to run a new container, you can use the Docker CLI to interact with the Docker daemon, which checks whether the specified image is already present on the local server. If not, it will find the image on a registry, download it, and then use it to run a container. The workflow, again, is shown in figure 2.7.

Docker on macOS and Windows. How does it work?

In the previous chapter, you learned that containers share the same operating system kernel and rely on Linux features such as namespaces and cgroups. We'll use Docker to run Spring Boot applications in Linux containers, but how is Docker supposed to work on a macOS or Windows machine?

When you install Docker on a Linux operating system, you get the full Docker Engine on your Linux host. However, if you install either *Docker Desktop for Mac* or *Docker Desktop for Windows*, only the Docker client is installed on your macOS/Windows host. Under the hood, a lightweight virtual machine is configured with Linux, and the Docker server component is installed on that machine. As a user, you will get almost the same experience as on a Linux machine; you will hardly notice any difference. But in reality, whenever you use the Docker CLI to perform operations, you are actually interacting with a Docker server on a different machine (the virtual machine running Linux).

You can verify this yourself by starting Docker and running the `docker version` command. You'll notice that the Docker client is running on a `darwin/amd64` architecture (on macOS) or on `windows/amd64` (on Windows), while the Docker server is running on `linux/amd64`.

```
$ docker version
Client:
 Cloud integration: v1.0.24
 Version:           20.10.14
 API version:       1.41
 Go version:        go1.16.15
 Git commit:        a224086
 Built:             Thu Mar 24 01:49:20 2022
 OS/Arch:           darwin/amd64
 Context:           default
 Experimental:      true

Server:
 Engine:
  Version:          20.10.14
  API version:      1.41 (minimum version 1.12)
  Go version:       go1.16.15
  Git commit:       87a90dc
  Built:            Thu Mar 24 01:45:44 2022
  OS/Arch:          linux/amd64
  Experimental:     false
```

Docker provides support for architectures other than AMD64. For example, if you use a MacBook with Apple Silicon (ARM-based chips), you'll find your Docker client running on a `darwin/arm64` architecture and the Docker server on `linux/arm64`.

3.2 Running a Spring application as a container

Let's go back to Catalog Service and see how you can run it as a container. There are different ways to achieve that, but here you will use the out-of-the-box integration of Spring Boot with Cloud Native Buildpacks (**https://buildpacks.io**), a project initiated by Heroku and Pivotal and now hosted by the CNCF. It provides a high-level abstraction for automatically transforming application source code into container images instead of using a low-level Dockerfile.

The Paketo Buildpacks (an implementation of the Cloud Native Buildpacks specification) are fully integrated with the Spring Boot Plugin, both for Gradle and Maven. This means that you can containerize your Spring Boot applications without downloading any additional tools, providing any extra dependencies, or writing a Dockerfile.

Chapter 6 will describe how the Cloud Native Buildpacks project works and how you can configure it to containerize your Spring Boot applications. For now, I'll give you a little preview of its features.

Open a Terminal window, navigate to the root folder of your Catalog Service project (catalog-service), and run the bootBuildImage Gradle task. That's all you need to do to package your application as a container image, using Cloud Native Buildpacks under the hood.

```
$ ./gradlew bootBuildImage
```

WARNING As I write this, the Paketo project is working on adding support for ARM64 images. You can follow the feature's progress on the Paketo Buildpacks project on GitHub: **https://github.com/paketo-buildpacks/stacks/issues/51**. Until that's complete, you can still use Buildpacks to build containers and run them via Docker Desktop on Apple Silicon computers, but the build process and the application startup phase will be slower than usual. Until official support is added, you can alternatively use the following command, which points to an experimental version of Paketo Buildpacks with ARM64 support: ./gradlew bootBuildImage --builder ghcr.io/thomasvitale/java -builder-arm64. Be aware that it's experimental and not ready for production. For more information, you can refer to the documentation on GitHub: **https://github.com/ThomasVitale/paketo-arm64**.

The first time you run the task, it will take a minute to download the packages used by Buildpacks to create the container image. The second time it will take only a few sec-

onds. The resulting image will be named catalog-service:0.0.1-SNAPSHOT by default (<project_name>:<version>). You can run the following command to get the details of the newly created image:

```
$ docker images catalog-service:0.0.1-SNAPSHOT
REPOSITORY        TAG              IMAGE ID       CREATED        SIZE
catalog-service   0.0.1-SNAPSHOT   f0247a113eff   42 years ago   275MB
```

NOTE You might have noticed in the previous command's output that it looks like the image was created 42 years ago. That's a convention used by Cloud Native Buildpacks to achieve reproducible builds. Subsequent execution of the build command should give the same output if nothing is changed in the input. Using an accurate creation timestamp would make that impossible, so Cloud Native Buildpacks uses a conventional timestamp (January 1, 1980).

The last thing to do is run the image and verify that the containerized application is working correctly. Open a Terminal window and run the following command:

```
$ docker run --rm --name catalog-service -p 8080:8080 \
    catalog-service:0.0.1-SNAPSHOT
```

WARNING If you're running the container on an Apple Silicon computer, the previous command might return a message like "WARNING: The requested image's platform (linux/amd64) does not match the detected host platform (linux/arm64/v8) and no specific platform was requested." In that case, you'll need to include an additional argument to the preceding command (before the image name) until support for ARM64 is added to Paketo Buildpacks: --platform linux/amd64.

You can refer to figure 2.8 for a description of the command.

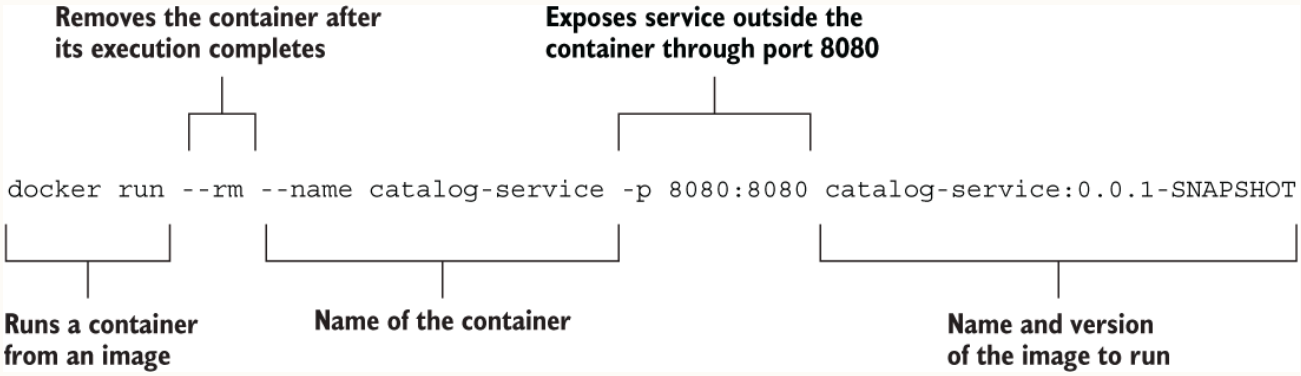


Figure 2.8 The Docker command to start a containerized application from an image

Open a browser window, navigate to http://localhost:8080/, and verify that you still get the same greeting you got earlier.

```
Welcome to the book catalog!
```

When you are done, stop the container with Ctrl-C.

In chapter 6, you'll learn more about how Docker works, how container images are built from a Spring Boot application, and how to use a container registry. I'll also show you how to use Docker Compose to manage containers instead of using the Docker CLI.

2.4 Managing containers with Kubernetes

So far, you have built a web application with Spring Boot (Catalog Service), containerized it with Cloud Native Buildpacks, and run it with Docker. To complete the pilot project for Polar Bookshop, you must perform one last step: deploy the application to a cloud environment. To do that, you will use Kubernetes, which has become the de facto standard for container orchestration. I will provide more details about Kubernetes in later chapters, but I want you to have a first taste of how it works and how you can use it to deploy a web application.

Kubernetes (often shortened as *K8s*) is an open source system for automating the deployment, scaling, and management of containerized applications (**https://kubernetes.io**). When you are working with containers in Docker, your deployment target is a machine. In the example from the previous section, it was your computer. In other scenarios it might be a virtual machine (VM). In any case, it's about deploying containers to a specific machine. However, when it comes to deploying containers without downtime, scaling them by leveraging the cloud's elasticity, or connecting them across different hosts, you'll need something more than a container engine. Instead of deploying to a specific machine, you are deploying to a cluster of machines, and Kubernetes, among other things, manages a cluster of machines for you. I covered this distinction in the previous chapter in the context of topologies. Figure 2.9 will remind you of the different deployment targets in a container topology and an orchestration topology.

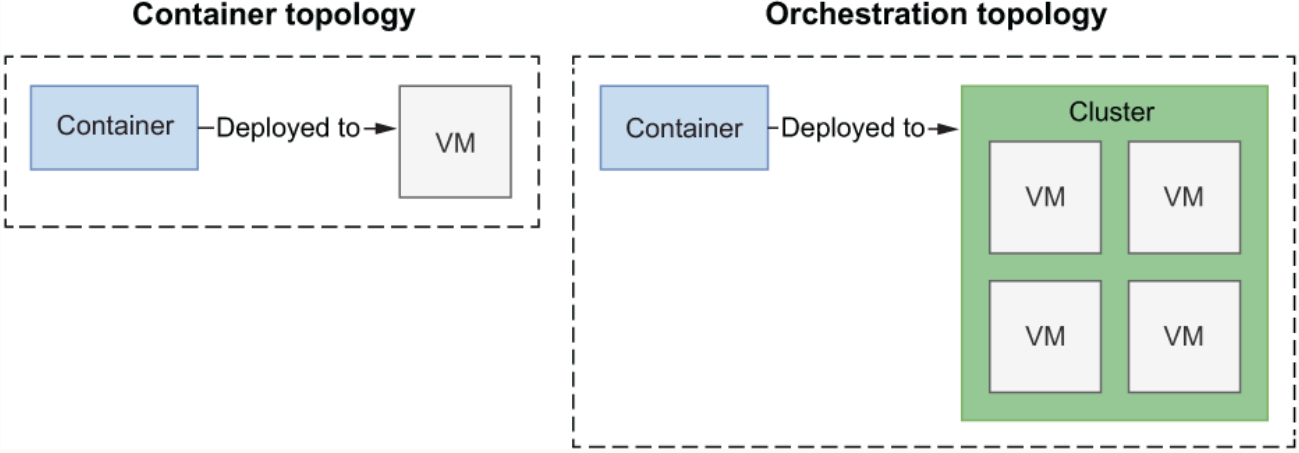


Figure 2.9 The deployment target of containers is a machine, whereas for orchestrators it's a cluster.

Before moving on, follow the instructions in section A.3 of appendix A to install minikube and set up a Kubernetes cluster in your local development environment. Once you are done with the installation procedure, you can start a local Kubernetes cluster with the following command:

```
$ minikube start
```

2.4.1 Introducing Kubernetes: Deployments, Pods, and Services

Kubernetes is an open source container orchestrator hosted by the CNCF. In just a few years, it has become the most used solution for container orchestration, and all the major cloud providers have offerings for Kubernetes as a service. Kubernetes can run on a desktop, on a data center on premises, in the cloud, and even on an IoT device.

When using a container topology, you need a machine with a container runtime. With Kubernetes, however, you switch to an orchestration topology, meaning that you need a cluster. A Kubernetes *cluster* is a set of worker machines (*nodes*) that run containerized applications. Every cluster has at least one worker node. With minikube. you can easily create a single-node cluster on your local machine. In production, you'll use a cluster managed by a cloud provider.

A Kubernetes cluster comprises machines called *worker nodes* on which your containerized applications are deployed. They provide capacity such as CPU, memory, network, and storage so that the containers can run and connect to a network.

The *control plane* is the container orchestration layer that manages the worker nodes. It exposes the API and interfaces to define, deploy, and manage the life cycle of contain-

ers. It offers all the essential elements that implement the typical features of an orchestrator, like cluster management, scheduling, and health monitoring.

NOTE In the context of container orchestration, *scheduling* means matching a container instance with a node where it will be run. The matching is based on a set of criteria, including the availability of enough computational resources to run the container on the node.

You can interact with Kubernetes through a CLI client, kubectl, which communicates with the control plane to perform some operations on the worker nodes. A client doesn't interact with the worker nodes directly. Figure 2.10 shows the high-level components of the Kubernetes architecture.

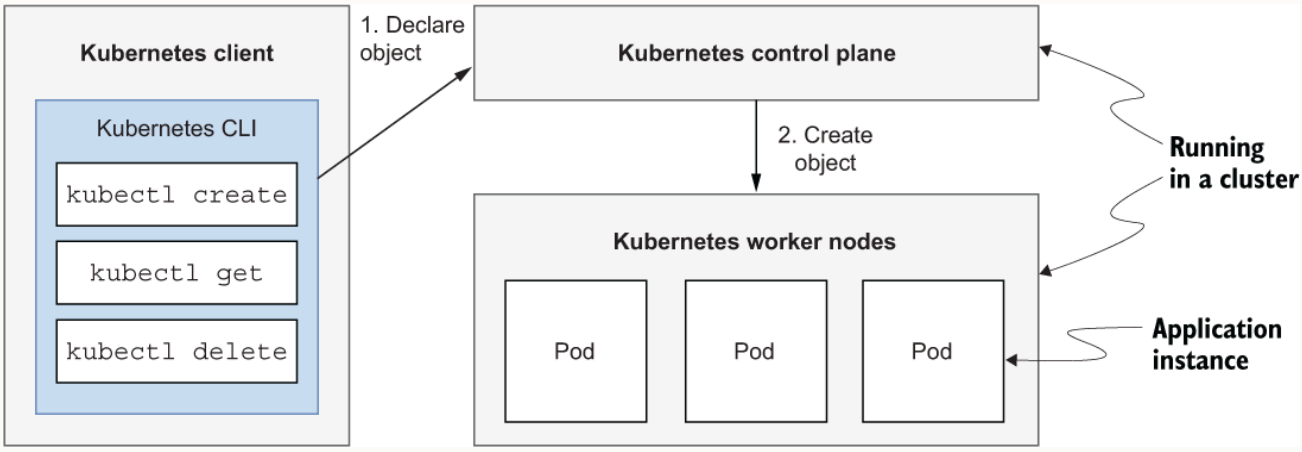


Figure 2.10 Kubernetes' main components are the API, the control plane, and the worker nodes.

Kubernetes can manage many different objects, either built-in or custom. In this section, you will use Pods, Deployments, and Services.

- *Pod*—The smallest deployable unit, which can include one or more containers. A Pod usually contains only one of your applications. It might also include extra containers supporting the primary application (such as containers providing additional functionality like logging or administrative tasks to run during the initialization step). Kubernetes manages Pods rather than containers directly.
- *Deployment*—A Deployment informs Kubernetes about the desired deployment state for your application. For each instance, it creates a Pod and keeps it healthy. Among other things, a Deployment allows you to manage Pods as a set.
- *Service*—A Deployment (a set of Pods) can be exposed to other nodes in the cluster or outside by defining a Service that also takes care of balancing the load between Pod instances.

NOTE Throughout the book, I will write Kubernetes resources with capital letters to distinguish them from when the same terms are used with different meanings. For example, I'll use *service* when referring to an application, whereas I'll write *Service* when I mean the Kubernetes object.

When you want to run a new application, you can define a *resource manifest*, a file that describes the desired state for the application. For example, you might specify that it should be replicated five times and exposed to the outside world through port 8080. Resource manifests are usually written using YAML. You can then use the kubectl client to ask the control plane to create the resources described by the manifest. In the end, the control plane processes the request using its internal components and creates the resources in the worker nodes. The control plane still relies on a container registry to fetch the image defined in the resource manifest. The workflow, again, is shown in figure 2.10.

4.2 Running a Spring application on Kubernetes

Let's go back to the Polar Bookshop project. In the previous section, you containerized the Catalog Service application. Now it's time to deploy it to a cluster using Kubernetes. You already have a cluster up and running in your local environment. What you need is a resource manifest.

The standard way to interact with Kubernetes is through declarative instructions that you can define in a YAML or JSON file. I will show you how to write resource manifests in chapter 7. Until then, you can use the Kubernetes CLI as you did with Docker earlier.

First you need to tell Kubernetes to deploy Catalog Service from a container image. You built one previously (catalog-service:0.0.1-SNAPSHOT). By default, minikube uses the Docker Hub registry to pull images, and it doesn't have access to your local ones. Therefore, it will not find the image you built for the Catalog Service application. But don't worry: you can manually import it into your local cluster.

Open a Terminal window, and run the following command:

```
$ minikube image load catalog-service:0.0.1-SNAPSHOT
```

The deployment unit will be a Pod, but you will not manage Pods directly. Instead, you'll want to let Kubernetes handle that. Pods are application instances, and as such they are ephemeral. To achieve the cloud native goals, you want the platform to take care of instantiating Pods so that if one goes down, it can be replaced by another one. What you

need is a *Deployment* resource that will make Kubernetes create application instances as *Pod* resources.

From a Terminal window, run the following command:

```
$ kubectl create deployment catalog-service \
    --image=catalog-service:0.0.1-SNAPSHOT
```

You can refer to figure 2.11 for a description of the command.

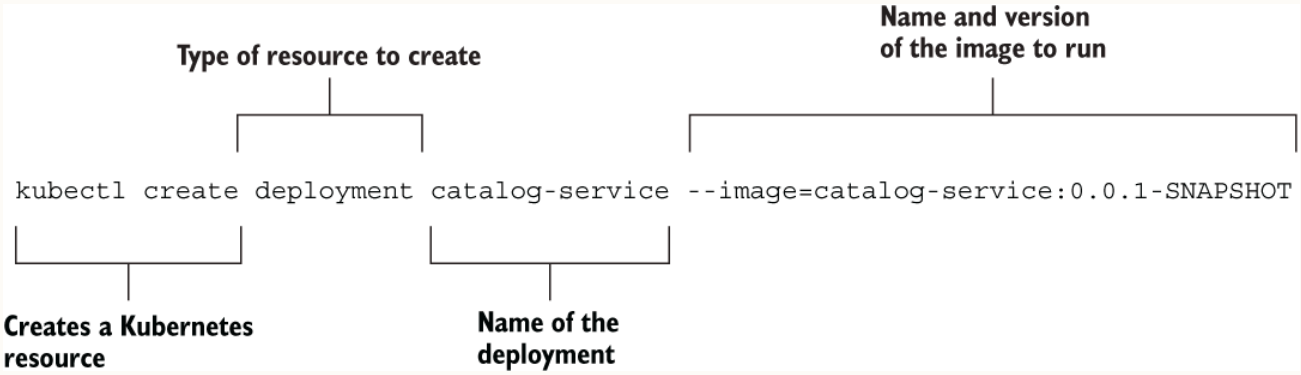


Figure 2.11 The Kubernetes command to create a Deployment from a container image. Kubernetes will take care of creating Pods for the application.

You can verify the creation of the Deployment object as follows:

```
$ kubectl get deployment
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
catalog-service   1/1     1            1           7s
```

Behind the scenes, Kubernetes created a Pod for the application defined in the Deployment resource. You can verify the creation of the Pod object as follows:

```
$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
catalog-service-5b9c996675-nzbhd    1/1     Running   0          21s
```

TIP You can check the application logs by running kubectl logs deployment/ catalog-service.

By default, applications running in Kubernetes are not accessible. Let's fix that. First, you can expose Catalog Service to the cluster through a Service resource by running the fol-

lowing command:

```
$ kubectl expose deployment catalog-service \
  --name=catalog-service \
  --port=8080
```

Figure 2.12 provides a description of the command.

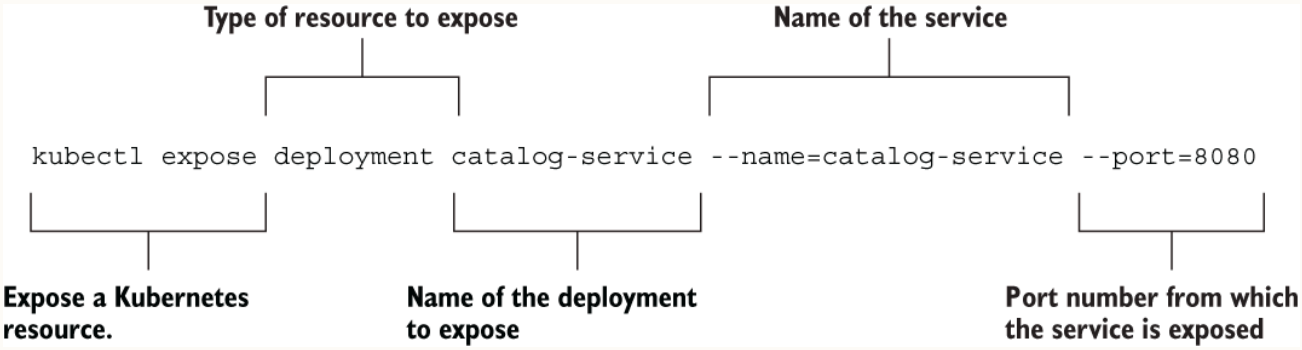


Figure 2.12 The Kubernetes command to expose a Deployment as a Service. The Catalog Service application will be exposed to the cluster network through port 8080.

The Service object exposes the application to other components inside the cluster. You can verify that it's been created correctly with the following command:

```
$ kubectl get service catalog-service
NAME           TYPE      CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
catalog-service   ClusterIP   10.96.141.159   <none>      8080/TCP   7s
```

You can then forward the traffic from a local port on your computer (for example, 8000) to the port exposed by the Service inside the cluster (8080). Remember the port mapping in Docker? This works in a similar way. The output of the command will tell you if the port forwarding is configured correctly:

```
$ kubectl port-forward service/catalog-service 8000:8080
Forwarding from 127.0.0.1:8000 -> 8080
Forwarding from [::1]:8000 -> 8080
```

You can refer to figure 2.13 for a description of the command.

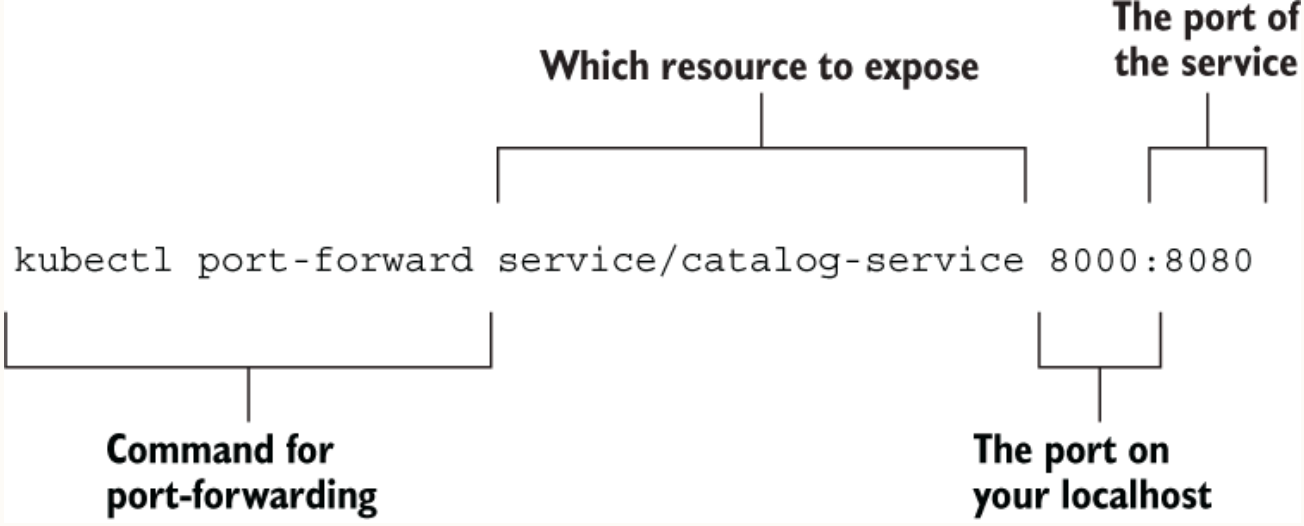


Figure 2.13 The Kubernetes command for forwarding a port from your localhost to a Service inside the cluster. The Catalog Service application will be exposed to your localhost through port 8000.

Now whenever you access port 8000 on your localhost, you will be forwarded to the Service inside the Kubernetes cluster responsible for exposing the Catalog Service application. Open a browser window, navigate to http://localhost:8000/ (make sure you use 8000 and not 8080), and verify that you still get the same greeting as before.

```
Welcome to the book catalog!
```

Well done! You started with a Java application implemented using Spring Boot. Then you containerized it with Cloud Native Buildpacks and ran it on Docker. Finally, you used Kubernetes to get the application deployed to a cluster. Sure, it was a local cluster, but it could have been a remote one in the cloud. The beauty of this process is that it works the same way regardless of the environment. You could use the exact same approach to deploy Catalog Service to a cluster in a public cloud infrastructure. Isn't it great?

In chapter 7, you will work more with Kubernetes. For now, terminate the port-forwarding process with Ctrl-C, delete the Service with kubectl delete service catalog-service, and delete the Deployment with kubectl delete deployment catalog-service. Finally, you can stop the Kubernetes cluster with minikube stop.

2.5 Polar Bookshop: A cloud native application

In this book, I aim to provide real-world code examples as often as possible. Now that you have explored some of the key concepts and have tried your hand at building, con-

tainerizing, and deploying a Spring application, let's take on a slightly more complicated project: an online bookshop. In the rest of the book, I'll guide you through developing a complete cloud native system based on Spring applications, containerizing it, and deploying it with Kubernetes in a public cloud.

For each concept we cover in the following chapters, I'll show you how to apply it to a real-world cloud native scenario for a complete hands-on learning experience. Remember that all the code used in this book is available on the book's GitHub repository.

This section will define the requirements for the cloud native project we'll build, and it will describe its architecture. Then I'll outline the main technologies and patterns we'll use to implement it.

2.5.1 Understanding the requirements of the system

Polar Bookshop is a specialized bookshop whose mission is to spread knowledge and information about the North Pole and the Arctic, where the bookshop is located: the Arctic's history, geography, animals, and so on. The organization managing the bookshop, Polarsophia, has decided to start selling its books online to spread them worldwide, but that is only the beginning. The project is very ambitious, and the vision includes a suite of software products to fulfill the Polarsophia mission. After the successful pilot project earlier in this chapter, the organization has decided to embark on a cloud native journey.

Throughout the book, you'll build the core part of a system with endless possibilities in terms of functionality and integration. The management plans to deliver new features in short iterations, reducing the time to market and getting early feedback from users. Their goal is to bring the bookshop close to everyone, everywhere, so the application should be highly scalable. With a worldwide audience, such a system needs to be highly available, so resilience is essential.

Polarsophia is a small organization, and they need to optimize costs, especially those regarding infrastructure. They can't afford to build their own data center, so they have decided to rent IT hardware from a third party.

By now, you can probably recognize some of the reasons why companies move to the cloud. That's what we'll do for the Polar Bookshop application. Of course, it will be a cloud native application.

Books will be available for sale through the application. When a customer purchases a book, they should be able to check on the status of their order. Two categories of people will use the Polar Bookshop application:

- Customers can browse books in the catalog, buy some, and check their orders.
- Employees can manage books, update existing ones, and add new items to the catalog.

Figure 2.14 describes the architecture of the Polar Bookshop cloud native system. As you can see, it's made up of several services. Some will implement the business logic of the system to provide the functionality already mentioned. Other services will implement shared concerns like centralized configuration. For clarity, the figure doesn't show the services responsible for the security and observability concerns. You'll learn about them later in the book.

In the following chapters, I'll guide you through figure 2.14 in more detail, adding more information about specific services and adopting different points of view to visualize the system in its deployment stage. For now, let's look at the patterns and technologies we'll use in the project.

2.5.2 Exploring patterns and technologies used in the project

I'll show you how to apply specific technologies or patterns to the Polar Bookshop project when I introduce each new topic in the book. Here I'll give you an overview of the main concerns we will address and the technologies and patterns we'll use to accomplish them.

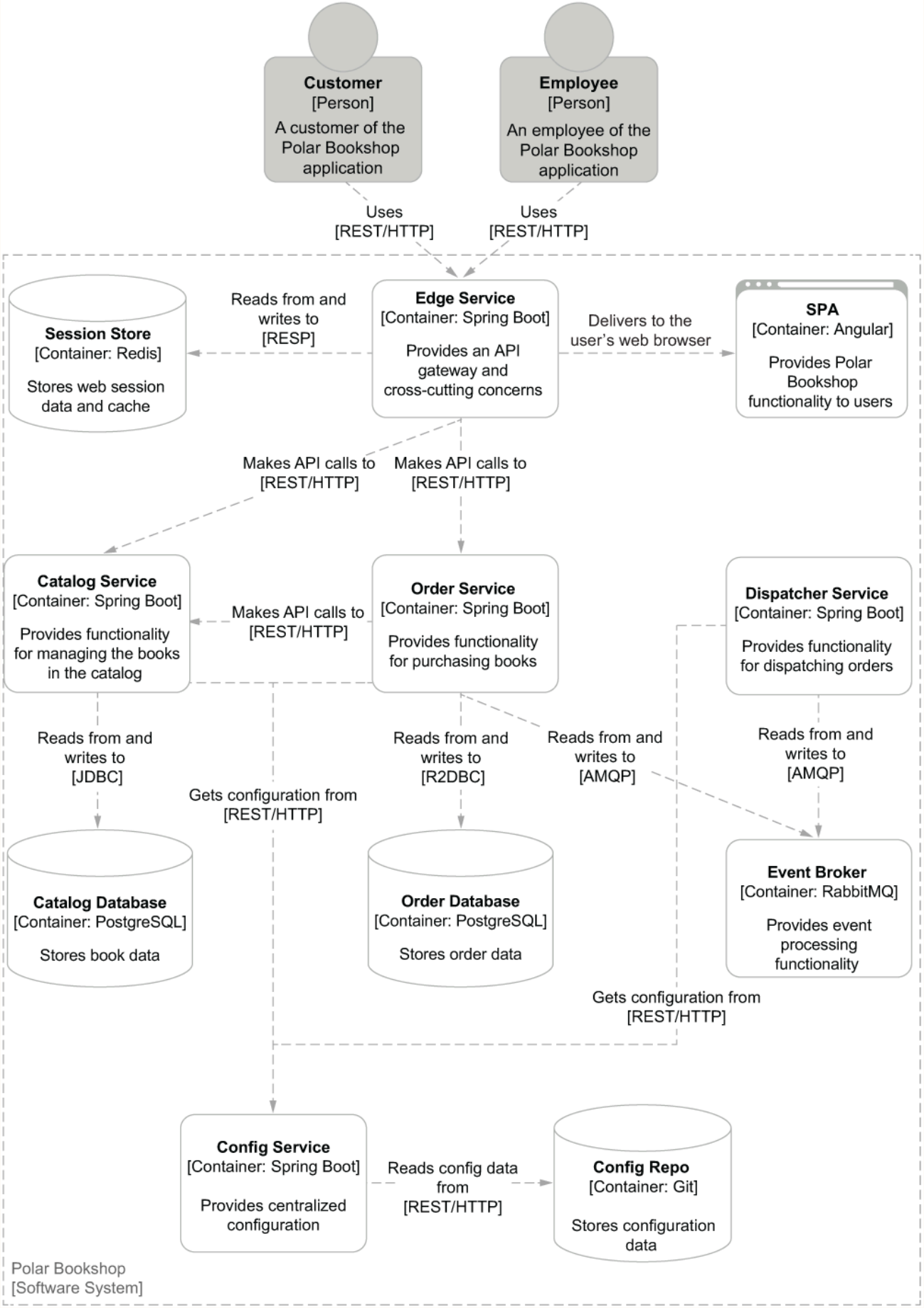


Figure 2.14 Architecture of Polar Bookshop. The cloud native system comprises application and data services with different responsibilities. For clarity, the security and observability services are not shown.

WEB AND INTERACTIONS

Polar Bookshop comprises several services that will have to communicate with each other to provide their functionality. You'll build RESTful services that interact synchronously over HTTP, both in a blocking way (using traditional servlets) and non-blocking ways (using reactive programming). Spring MVC and Spring WebFlux (on top of Project Reactor) will be your main tools for accomplishing such a result.

When building cloud native applications, you should design loosely coupled services and consider how you can keep data consistent in a distributed system context. Synchronous communications can generate problems when multiple services are involved in accomplishing a function. That's why event-driven programming is getting very popular in the cloud: it allows you to overcome the issues of synchronous communications.

I'll show you how to use events and messaging systems to decouple services and ensure data consistency. You'll use Spring Cloud Stream to implement data streams between services and Spring Cloud Function to define message handlers as functions. The latter approach could naturally evolve in serverless applications deployed to platforms like Azure Functions, AWS Lambda, or Knative.

DATA

Data is a crucial part of software systems. In Polar Bookshop you'll use the PostgreSQL relational database to permanently store the data processed by the application. I'll show you how to integrate an application with a data source using Spring Data JDBC (imperative) and Spring Data R2DBC (reactive). You'll then learn how to evolve a data source and manage schema migrations with Flyway.

Cloud native applications should be stateless, but the state needs to be stored somewhere. In Polar Bookshop you'll use Redis to externalize the session storage to a data store and keep applications stateless and scalable. Spring Session makes it easy to implement clustered user sessions. In particular, I'll show you how to use Spring Session Data Redis to integrate the application session management with Redis.

Besides persisted data and session data, you'll also deal with messages to implement event-driven programming patterns. Spring AMQP and RabbitMQ will be the technologies you'll use to do so.

Locally, you'll run these data services in Docker containers. In production, you'll rely on the managed services offered by a cloud provider like DigitalOcean or Azure, which take care of critical concerns like high availability, clustering, storage, and data replication.

CONFIGURATION

Throughout the book I'll show you how to configure the services in Polar Bookshop in different ways. I'll start by exploring the options offered by Spring Boot properties and profiles, and when to use them. You'll then learn how to apply external configuration by using environment variables when running a Spring application as a JAR and as a container. Then you'll see how to centralize the configuration management through a configuration server with Spring Cloud Config. Finally, I'll teach you how to use ConfigMaps and Secrets in Kubernetes.

ROUTING

Polar Bookshop, being a distributed system, will need some routing configuration. Kubernetes has a built-in service discovery feature that can help you decouple services from their physical addresses and hostnames. Cloud native applications are scalable, so any interactions between them should take that into account: which instance should you call? Once again, Kubernetes offers you a load-balancing feature natively, so you don't need to implement anything in your application.

Using Spring Cloud Gateway, I'll guide you through implementing a service that will act as an API gateway to shield the outside from any internal API changes. It will also be an edge service that you'll use to address cross-cutting concerns, like security and resilience, in one place. Such a service will be the entry point to Polar Bookshop, and it will have to be highly available, performant, and fault-tolerant.

OBSERVABILITY

The services in the Polar Bookshop system should be observable to be defined as cloud native. I'll show you how to use Spring Boot Actuator to set up health and info endpoints and expose metrics with Micrometer to be fetched and processed by Prometheus. You'll then use Grafana to visualize the most critical metrics in informative dashboards.

Requests can be handled by more than one service, so you'll need distributed tracing functionality to follow the request flow from one service to another. You'll set that up with OpenTelemetry. Then Grafana Tempo will fetch, process, and visualize the traces to give you a complete picture of how the system accomplishes its functions.

Finally you'll need a logging strategy in place. We should handle logs as event streams, so you'll make your Spring applications stream log events to the standard output without considering how they are processed or stored. Fluent Bit will take care of collecting

logs from all services, Loki will store and process them, and Grafana will let you browse them.

RESILIENCE

Cloud native applications should be resilient. For the Polar Bookshop project, I'll show you various techniques for making an application resilient using Project Reactor, Spring Cloud Circuit Breaker, and Resilience4J to implement circuit breakers, retries, timeouts, and other patterns.

SECURITY

Security is a vast subject that I won't be able to cover deeply in this book. Still, I recommend exploring the topic, since it's one of the most critical software concerns nowadays. It's a pervasive concern that should be addressed continuously from the very beginning of the project.

For Polar Bookshop, I'll show you how to add authentication and authorization functionality to cloud native applications. You'll see how to secure communications between services, and between users and applications. OAuth 2.1 and OpenID Connect will be the standards you'll rely on to implement such functionality. Spring Security supports those standards and integrates seamlessly with external services to provide authentication and authorization. You'll use Keycloak for identity and access control management.

Also, I'll introduce the concepts of secrets management and encryption. I won't be able to go too deep into those topics, but I'll show you how to manage secrets to configure Spring Boot applications.

TESTING

Automated testing is paramount for the success of a cloud native application. A few levels of automated tests will cover the Polar Bookshop application. I'll show you how to use JUnit5 to write unit tests. Spring Boot adds many convenient utilities that improve integration testing, and you will use them to ensure the quality of your services. You'll write tests for the various features used in Polar Bookshop, including REST endpoints, messaging streams, data integrations, and security.

Keeping parity across environments is essential to ensure the quality of your applications. This is especially true when it comes to backing services. In production, you will use services such as PostgreSQL and Redis. During testing, you should use similar services rather than mocks or test-specific tools like the H2 in-memory database. The

Testcontainers framework will help you use real services as containers in your automated tests.

BUILD AND DEPLOYMENT

Polar Bookshop's primary services will use Spring. You'll see how to package a Spring application, run it as a JAR file, containerize it with Cloud Native Buildpacks, run it with Docker, and finally deploy containers with Kubernetes. You'll also see how to compile Spring applications to native images using Spring Native and GraalVM, and use them in serverless architectures, taking advantage of their instant startup time, instant peak performance, reduced memory consumption, and reduced image size. Then you'll deploy them on a serverless platform built on top of Kubernetes with Knative.

I'll show you how to automate the build stage by setting up a deployment pipeline with GitHub Actions. The pipeline will build the application at every commit, run the tests, and package it ready for deployment. Such automation will support a continuous delivery culture to bring value to customers quickly and reliably. In the end, you'll also automate the deployment of Polar Bookshop to the production Kubernetes cluster using GitOps practices and Argo CD.

UI

This book is focused on backend technologies, so I won't teach you any frontend subjects. Of course, your application will need a frontend for the users to interact with. In the case of Polar Bookshop, you'll rely on a client application using the Angular framework. I won't show you the UI application code in this book because it's out of scope, but I've included it in the code repository accompanying the book.

Summary

- The 15-Factor methodology identifies development principles for building applications that offer maximum portability across execution environments, are suitable to be deployed on cloud platforms, can be scaled, guarantee environment parity between development and production, and enable continuous delivery.
- Spring is a suite of projects that provide all of the most common functionality for building modern applications in Java.
- Spring Framework provides an application context in which beans and properties are managed throughout the entire life cycle.

- Spring Boot lays the foundation for cloud native development by speeding up the building of production-ready applications, including embedded servers, auto-configuration, monitoring, and containerization features.
- Container images are lightweight executable packages that include everything needed to run the applications inside.
- Docker is an OCI-compliant platform for building and running containers.
- A Spring Boot application can be packaged as a container image with Cloud Native Buildpacks, a CNCF project that specifies how to convert application source code into production-ready container images.
- When dealing with several containers, as is usually the case in a cloud native system, you need to manage this complex system. Kubernetes provides functionality to orchestrate, schedule, and manage containers.
- Kubernetes Pods are the minimum deployment units.
- Kubernetes Deployments describe how to create application instances as Pods, starting from container images.
- Kubernetes Services allow you to expose application endpoints outside the cluster.

---

1. A. Wiggins, "The Twelve-Factor App," **https://12factor.net**.

2. K. Hoffman, *Beyond the Twelve-Factor App* (O'Reilly, 2016).

Previous chapter

< 1 Introduction to cloud native

Next chapter

Part 2 Cloud native development >